2주차 4일차: ObservableCollection과 표 갱신 — 120분

1. 학습 목표 — 5분

- 행 추가 알림과 행 내부 값 변경 알림을 구분합니다.
- 입력값으로 첫 행의 제목을 바꾸고 새 행을 추가합니다.
- 바인딩으로 입력부터 결과 표까지 연결합니다.

시작 상태는 day07입니다. 두 동작은 알림을 관찰하기 위한 학습용이며, 최종 API 프로그램에서는 서버 응답으로 표를 채웁니다.

2. 핵심 개념 설명 — 15분

List<Post>에 처음 들어 있던 두 행은 잘 표시됐습니다. 하지만 화면이 이미 연결된 뒤 목록에 항목을 더하거나 지우는 사실도 알려야 합니다.

ObservableCollection<Post>는 항목 추가·삭제 같은 목록 변화를 알리는 컬렉션입니다. Posts.Add를 호출하면 DataGrid에 새 행이 나타납니다.

컬렉션이 알리는 것과 항목이 알리는 것은 다릅니다. 기존 Post의 Title 값이 바뀌는 것은 목록에 행을 추가한 일이 아닙니다. 그래서 Post도 INotifyPropertyChanged를 구현하여 Title 변경을 알립니다.

바뀌는 것	알림을 맡는 곳
결과·상태 문구	MainWindow의 INotifyPropertyChanged
Posts에 행 추가·삭제	ObservableCollection<Post>
기존 행의 제목	Post의 INotifyPropertyChanged

Posts는 get만 있지만 Posts.Add는 가능합니다. "목록 객체를 다른 것으로 교체"하는 것과 "같은 목록 안에 항목을 추가"하는 것은 다릅니다. 오늘은 목록 객체를 계속 유지합니다.

실무·면접 기초: ObservableCollection을 쓰더라도 항목 속성 변경까지 자동으로 알려 주는 것은 아닙니다. 데이터 변경의 위치를 먼저 구분하세요.

3. 짧고 이해하기 쉬운 코드 예제 — 15분

```
public ObservableCollection<Post> Posts { get; } = new()
{
    new Post { Id = 1, Title = "첫 번째 예제 게시글" },
    new Post { Id = 2, Title = "두 번째 예제 게시글" }
};
```

버튼 처리에서 목록과 항목을 각각 바꿉니다.

```
        Posts[0].Title = text;
        Posts.Add(new Post { Id = Posts.Count + 1, Title = text });
```

Post의 Title에는 이전 시간과 같은 속성 변경 알림을 넣습니다.

```
    private string title = "";
    public string Title
    {
        get => title;
        set
        {
            title = value;
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(nameof(Title)));
        }
    }
```

번호는 학습용으로 현재 개수에 1을 더합니다. 실제 데이터베이스의 기본키 생성 규칙을 구현하는 예제는 아닙니다.

오늘 바뀌는 파일입니다. 링크에는 생략 없는 전체 코드가 있습니다.

파일	전체 코드
Post.cs	day08의 Post.cs
MainWindow.xaml.cs	day08의 MainWindow.xaml.cs

본문에는 핵심 부분만 보여 줍니다. 파일을 만들거나 교체할 때는 위 링크의 **전체 코드**를 사용하세요.
App 파일은 바뀌지 않습니다.

4. Visual Studio에서 따라 하는 실습 — 75분

파일 적용과 실행 — 25분

1. MainWindow.xaml.cs와 Post.cs를 오늘의 전체 코드로 교체합니다. XAML은 day07과 같습니다.
2. MainWindow의 Posts 타입이 ObservableCollection<Post>로 바뀐 것을 봅니다.
3. Post.cs의 Title에 값 저장과 PropertyChanged 호출이 있는지 확인합니다.
4. F5로 실행해 초기 두 행이 보이는지 확인합니다.

두 종류의 변경 관찰 — 25분

1. 입력창에 "변경한 제목"을 넣고 실행합니다. 첫 행 제목이 바뀌고 번호 3인 행이 생깁니다.
2. 입력을 "다음 제목"으로 바꾸고 다시 실행합니다. 첫 행은 새 제목으로 바뀌고 번호 4인 행이 생깁니다.
3. 빈 입력을 실행합니다. 행은 추가되지 않고 입력 안내만 나타납니다.
4. 프로그램을 닫고 다시 실행합니다. 저장 기능이 없으므로 초기 두 행으로 돌아옵니다.

알림의 차이를 코드에서 확인 — 15분

1. Run의 Posts[0].Title 대입문과 Post.cs의 Title set 안 title=value 줄에 각각 중단점을 겁니다.

2. 입력 후 실행하고 첫 중단점에서 F5를 누릅니다. Title의 set 안에서 멈추면 F10으로 값 저장과 속성 알림을 봅니다.
3. F5로 계속 실행합니다. Posts.Add는 컬렉션이 추가 알림을 맡으므로 우리가 별도의 Notify(nameof(Posts))를 호출하지 않았다는 점을 확인합니다.
4. 중단점을 제거하고 실행을 종료합니다.

최종 저장 — 10분

1. 다시 실행해 입력 → 클릭 → 첫 행 변경·새 행 추가를 한 번 이어서 확인합니다.
2. 코드와 화면에서 "속성 하나", "목록 전체", "목록 안의 항목"을 각각 가리켜 봅니다. 별도 문제 풀이 파일은 만들지 않습니다.
3. 저장하고 종료합니다. 이 상태가 3주차 시작 코드입니다.

진도가 다르면 저장소 루트의 터미널에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 7로 시작 상태를 적용할 수 있습니다. 파일 복사가 어렵다면 같은 명령의 Stage를 8로 바꾸면 오늘 완료본이 적용됩니다. 실행 중인 앱을 먼저 닫고 Visual Studio에서 변경 파일을 다시 불러옵니다. 기존 교재 파일은 자동 백업됩니다. 단계 적용 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 10분

- 화면과 C# 데이터는 DataContext와 Binding으로 연결합니다.
- 변경을 저장하는 것과 변경 알림을 보내는 것은 구분합니다.
- ObservableCollection은 행의 추가·삭제를, Post의 INotifyPropertyChanged는 제목 변경을 알립니다.
- 표는 읽기 전용입니다. 이번 주에는 편집 기능이나 SQL 연결을 추가하지 않습니다.

완료본: day08. 다음: 3주차 1일차.